Cookie Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Cookie & Session Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use controlled accounts and synthetic data. Never copy another person's cookies, session IDs, or tokens, and never test destructive payloads against production.*

■ Executive Summary

Cookies often carry session identifiers or influence application state, so weak validation or session handling can lead to data exposure, cross-site scripting, privilege errors, or denial of service. The supplied three-page PDF covers sensitive cookie data, oversized cookies, arbitrary cookie values, parameter pollution, authentication checks, session lifecycle, privilege escalation, and injection risks.

■ Core Testing Methodology

1. **Inventory cookies.** Record names, purpose, domain/path scope, expiry, and security attributes using your own session.
2. **Check sensitive data.** Verify that cookies do not contain passwords, personal data, or unnecessary authorization details.
3. **Review cookie size limits.** Confirm that oversized cookies are rejected safely and cannot cause excessive resource use.
4. **Test trust boundaries.** Determine whether changing a client-side cookie can alter identity, role, organization, or entitlement.
5. **Review duplicate parameters.** Check whether repeated identity fields are handled unambiguously and whether authorization remains enforced.
6. **Review session lifecycle.** Test logout, password reset/change, expiry, and concurrent sessions with controlled accounts.
7. **Review output reflection.** If cookie values are displayed, verify context-appropriate output encoding.
8. **Review injection risks.** Inspect server-side code for unsafe use of cookie values in SQL, template evaluation, or OS commands; do not execute dangerous payloads against live systems.
9. **Check resource limits.** Ensure unusually large cookies and malformed values fail safely.
10. **Document and retest.** Redact cookie values and verify the fix across related endpoints.

■ High-Value Bug Bounty Tips & Edge Cases

- **Sensitive cookie data:** Keep session identifiers opaque and avoid storing unnecessary personal or authorization data in client-readable values.
- **Cookie length:** Enforce sensible size limits at the edge and application layers.
- **Arbitrary cookie injection:** Invalid cookie values should be rejected or ignored safely; they must not cause privilege changes or sensitive stack traces.
- **Mass assignment/parameter pollution:** Duplicate identity parameters can be interpreted inconsistently across layers. Define and enforce one schema.
- **Cookie bomb/DoS:** Excessive cookie sizes can increase parsing and proxy overhead. Test size handling with small, approved lab cases.
- **SQL injection:** Cookie values must be parameterized if used in database queries.

- **Authentication bypass:** Removing a cookie must not accidentally bypass checks; protected endpoints must require valid server-side session state.
- **XSS:** Reflected cookie data must be encoded for the output context.
- **Insufficient session management:** Review expiry, logout invalidation, password-change invalidation, and concurrent-session policy.
- **Privilege escalation:** Roles and organization membership should be derived from trusted server-side records, not editable cookie fields.
- **Session puzzling:** Avoid reusing one session variable for unrelated purposes or trusting ambiguous state.
- **Source limitation:** The PDF contains several dangerous injection examples. This rewrite describes the risk categories without reproducing code-execution payloads.

■■ Educational Checklists & Safe Examples

```
GET /account/preferences HTTP/1.1
Host: example.com
Cookie: session=[YOUR-TEST-SESSION]
```

- [ ] Cookies use `Secure`, `HttpOnly`, and appropriate `SameSite` settings.
- [ ] Session IDs are opaque and unpredictable.
- [ ] Server-side authorization does not rely on editable client fields.
- [ ] Logout and security events invalidate sessions as designed.
- [ ] Cookie size limits are enforced.
- [ ] Duplicate or malformed values are handled consistently.
- [ ] Cookie values are safely encoded and parameterized wherever used.
- [ ] Public evidence redacts session values.

■■ Mitigation & Remediation

- Store authoritative session and authorization state server-side.
- Use unpredictable session IDs and rotate them after authentication or privilege changes.
- Set `Secure`, `HttpOnly`, and suitable `SameSite` cookie attributes.
- Enforce size limits and reject malformed or ambiguous cookies.
- Parameterize database queries and never evaluate cookie data as code.
- Invalidate sessions after logout, password reset, and other risk events.
- Apply output encoding, CSRF protection, and least privilege.
- Avoid logging raw cookies or session tokens.

■ Source Notes

Based on the supplied three-page *cookie.pdf*. Page 1 covers sensitive data, cookie length, arbitrary values, mass assignment, cookie-bomb DoS, SQL injection, parameter pollution, and authentication checks. Page 2 covers XSS, session management, privilege escalation, session puzzling, and unsafe Python/OS command evaluation examples. The examples are not reproduced as operational payloads.

Educational use only. Keep tokens secret and test only accounts you control.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.